Day 3

Memory, state & real tools

The agent remembers you, and reads the docs.

Where we left off: two walls

1. **Forgets.** "Track Pixel Pirates for me" → new session → gone.
2. **Can't see the docs.** *"Did the devs fix the save bug?"* The answer is in `data/docs/`, the agent has reviews only.

Today: state ▸ failure design ▸ RAG ▸ callbacks.
Same analyst, upgraded organ by organ.

Part 1

Sessions & state

What a session actually is

```
session = events + state
          ──────   ─────
          the       a key-value dict
          transcript that YOUR CODE writes
```

Day 1 you learned: chat memory = resending history. True, but history is a *transcript*: unqueryable, unbounded, gone with the session.

State is memory *by design*: explicit keys, explicit writes, inspectable in the UI.

State keys have lifetimes

Key	Survives	Use for
`draft`	this session	workflow scratch *(Day 4 lives here)*
`user:tracked_games`	**the user, across sessions**	preferences, watchlists
`app:catalog_version`	the whole app	shared config
`temp:refund_blocked`	this turn	callback flags

The prefix is the design decision: *who* should remember this, *for how long?*

Tools get a secret parameter

```python
def track_game(game_id: str, tool_context: ToolContext) -> dict:
    watchlist = tool_context.state.get("user:tracked_games", [])
    ...
    tool_context.state["user:tracked_games"] = watchlist   # assignment persists
```

- ADK injects `tool_context`; **the model never sees it** (not in the schema)
- read with `.get(key, default)` · persist by **assignment**
- the same state, visible live in `adk web` → State panel

⌨️ Walkthrough: Part 1

- ✅ inspect a session: events + (empty) state
- ✅ implement `track_game` / `list_tracked_games`
- ✅ **new session** → it still knows your watchlist
- ★★ `untrack_game`: design the not-tracked case yourself

Break

10 minutes

Part 2

The tool ecosystem

yours · Google's · everyone else's

Every tool fails eventually

Databases go down. APIs rate-limit. Users pass garbage.

The question is never *if*. It's **who decides what happens next**:

Tool does	Who decides	Result
`raise RuntimeError(...)`	the model, alone	apology, retry… or a confident guess 🎲
`{"status": "error", "message": "...offline. Review stats still available."}`	**you**	model relays the problem *and the plan B you wrote*

Your error message is the model's script for the failure.
Write it like one.

Built-in tools: the other kind

Custom tools	Built-in tools
your code, your machine	Google's code, inside the model call
your data	the live world (`google_search`), sandboxes (code execution)
combine freely	restrictions: model support, combining limits

Rule of thumb so far: **your data → custom · the live world → built-in.**

One column missing…

The third kind: tools you didn't write

MCP, the Model Context Protocol. An open standard: a *server* exposes tools, any *client* consumes them (ADK, Claude, your IDE). Thousands of servers exist: filesystems, databases, GitHub, Slack…

```
McpToolset(
    connection_params=...,  # spawns: npx …server-filesystem data/docs
    tool_filter=["list_directory", "read_text_file", "search_files"],
)
```

- Day 2, full circle: **the schema is the interface**, and that's why a stranger's tools compose with your agent at all
- `tool_filter` drops `write_file`, `edit_file`, `move_file`: **a tool list is a permission list**

The taxonomy, complete

	runs	good for
custom tools	your code, your machine	your data, your rules
built-in tools	Google's, inside the call	the live world, sandboxes
MCP tools	anyone's server	everything someone already built

Choosing *which kind of tool* is now part of designing an agent,
same as choosing a library vs writing the function.

⌨️ Walkthrough: Part 2

- ✅ add `get_sales_data`, broken on purpose. Watch the flail
- ✅ fix it to fail *professionally*; compare the answers
- ✅ `search_demo`: ground the model in live Google Search
- ✅ `mcp_demo`: a filesystem MCP server. Read Events: whose tools are these?

Lunch

30 minutes · after: wall #2 falls

Part 3

A real RAG tool

You already know RAG. All of it.

Day 1: you embedded 300 reviews and searched them. **That was RAG's "R".**

Today, same 40 lines, new corpus: `data/docs/`:
store pages + **dated patch notes**. Reviews are what players *feel*;
docs are what's actually *true* and *when it shipped*.

```
index:  docs ──► embed ──► cache          (once)
query:  question ──► embed ──► cosine ──► top-k     (per call)
```

One embedding per file (small files). Big corpora add one step: **chunking**.

Two corpora, one discipline

The instruction now routes between sources:

- players **say / feel / complain** → `search_reviews`
- devs **shipped / fixed / changed** → `search_docs`
- *"did they fix X?"* → **both**, then compare the dates

That date comparison is Day 1's exercise 7.3, reviews vs patch notes, now performed by the agent on demand.

⌨️ Walkthrough: Part 3

- ✅ build the doc index (`python -m playfield_agent.retrieval`)
- ✅ implement `search_docs` (your third retrieval engine, and the last one, promise)
- ✅ the payoff: *"did they fix the save bug?"* → v1.2, January 2026, cited
- ★★ ask something the docs *don't* know: does it stay honest?

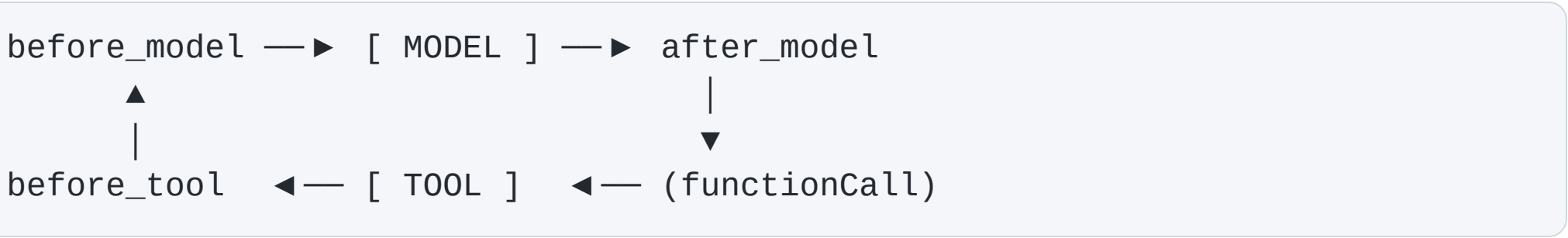
Break

10 minutes

Part 4

Callbacks: observe & guard

Hooks around the loop



Two return values, one rule:

- `None` → proceed normally
- **a value** → *replace* that step entirely (skip the model / skip the tool)

Same mechanism → two superpowers: **logging** and **guardrails**.

Guardrail: refunds never reach the model

```
def refund_guardrail(callback_context, llm_request):
    if looks_like_refund(last_user_message(llm_request)):
        return LlmResponse(content=...)   # canned policy answer
    return None                            # business as usual
```

Why before the *model*, not in the instruction?

- **zero tokens** spent, zero latency
- **deterministic**: a prompt can be argued with; a callback cannot
- auditable: `temp:refund_blocked = True`

Instructions persuade. **Callbacks enforce.**

⌨️ Walkthrough: Part 4

- ✅ `log_tool_calls`: every tool call in your terminal
- ✅ implement + wire the refund guardrail
- ✅ test: English, Romanian, and a legit question that sails through
- ★★ after_tool_callback: collect sources into state

Defense in depth (Day 5's checklist, previewed)

Layer	Catches
before_model callback	banned topics, before tokens are spent
before_tool callback	bad/dangerous arguments
the tool itself	failures → structured errors
the instruction	tone, scope, evidence rules

No single layer is trusted with everything.
That's not paranoia. That's how you ship.

Day 3, banked ✅

Your analyst now has:

- **durable memory**: watchlist that survives sessions (`user:` state)
- **two corpora**: reviews (feelings) + docs (facts), date-aware
- **honest failures**: error dicts with a plan B
- **observability**: every tool call logged
- **a hard guardrail**: refunds never reach the model

One agent. One instruction. Doing… a lot. 🤔

Next session

Day 4: Multi-agent systems

Ask it for a full report today. Note the "okay".
Next time, a team makes it good.